

Code Sentinel

A Multi-Agent Code Review Platform

Final Project Report

Live application: https://codesentinel.scribly.fun
API status: https://api.codesentinel.scribly.fun/healthz
Source code: https://github.com/sumit-0804/CodeSentinel

Team

Sumit Goyal – 202512092
Durgesh – 202512052
Tej Prakash Tak – 202512113
Nevil Nandasana – 202512009
Parin Makwana – 202512098
Ayush Soni – 202512030
Vatsal Kamani – 202512068
Moksh Mehta – 202512120

November 2026

Contents

1. Summary 2
What was delivered 2
2. System Overview 3
2.1 Architecture 3
2.2 How a review works 4
2.3 Data model 6
3. Technology Stack 7
4. Features 8
4.1 Pull request review 8
4.2 Dashboard 8
4.3 Operations 8
5. Challenges and Solutions 9
5.1 Staying inside free LLM quotas 9
5.2 Too little context from diffs 9
5.3 Duplicate findings from different agents 9
5.4 Running linters where Docker is not available 9
5.5 Free hosting that sleeps 10
5.6 Finding the real user behind proxies 10
5.7 Sign-out that takes effect immediately 10
5.8 Keeping eight people's work consistent 10
6. Screenshots 11
6.1 On GitHub 11
6.2 Dashboard 12
7. Testing 15
7.1 Approach 15
7.2 Results 15
7.3 Demo data 15
8. Deployment 16
8.1 Hosting 16
8.2 CI/CD 16
8.3 Security configuration 16
8.4 Monitoring 17
9. Team and Contributions 18
Limitations and future work 18
Conclusion 18

1. Summary

Code Sentinel reviews pull requests automatically. When a developer opens or updates a pull request on GitHub, five independent agents read the changed code in parallel: **Security**, **Style**, **Performance**, **Logic** and **Documentation**. An orchestrator collects their findings, merges the ones that describe the same problem, ranks them by severity and sends one combined review back to the pull request. The developer sees a GitHub Check Run, inline comments on the exact lines, and fixes that can be committed with one click.

A web dashboard sits next to the GitHub App. Teams sign in, connect repositories, choose which agents run and whether an agent may block a merge, follow a review while it runs, commit or dismiss fixes, and read the review history and an audit log.

What was delivered

- A working, deployed application: seven backend services, a Next.js dashboard and a PostgreSQL database, live at https://codesentinel.scribly.fun.
- All five agents, each as its own service behind one shared API contract.
- A GitHub App that reviews real pull requests end to end, including one-click and “apply all” fixes.
- Sign-in with email, GitHub or Google, optional two-factor authentication, roles, rate limits and an audit log.
- Monitoring with dashboards and alerts, and a CI pipeline that gates every deployment.
- 868 automated tests, all passing on Node.js 22 and 24, with commits from all eight team members.

The whole system runs on free hosting plans. Much of the engineering effort went into making that reliable: staying inside free LLM quotas, waking sleeping services in time, and answering GitHub’s webhooks within its ten-second limit. Section 5 describes these problems and how they were solved.

2. System Overview

2.1 Architecture

Code Sentinel is built as a service-oriented system. Each part is a small service with one job and a written API contract, so the parts can be developed, tested, deployed and scaled on their own.

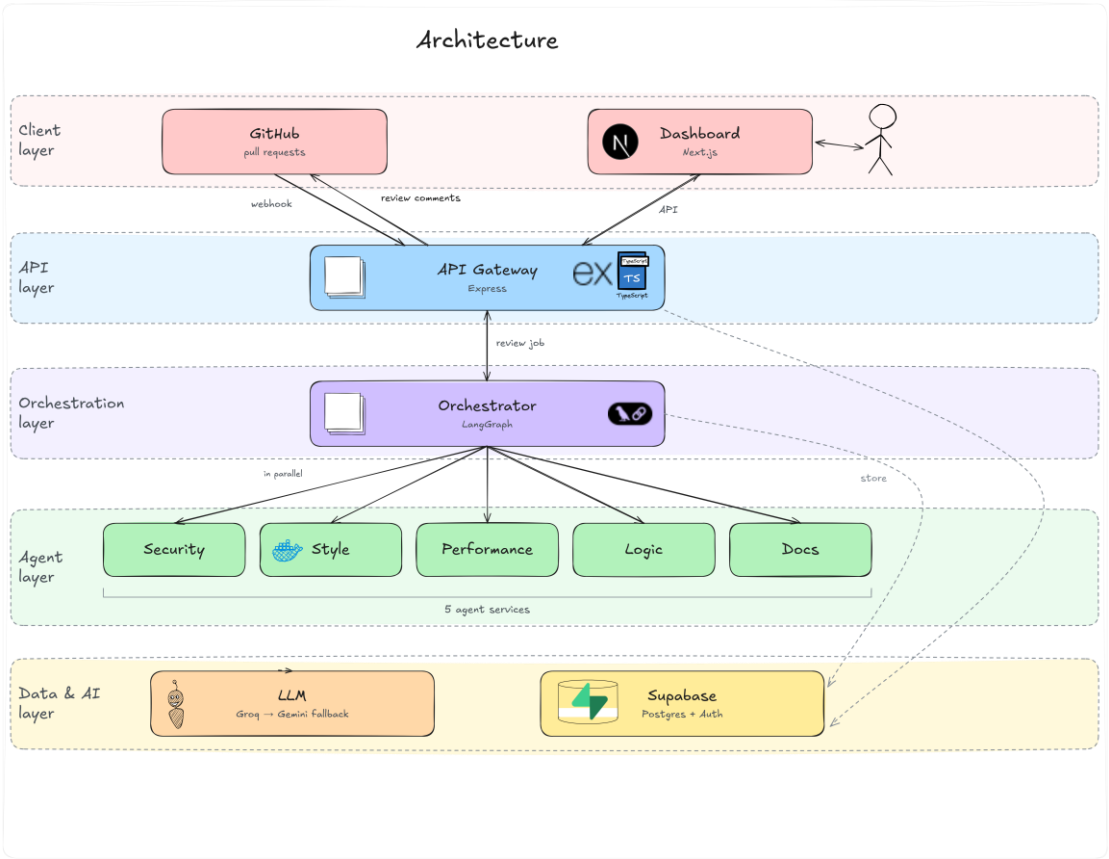


Figure 1. System architecture

Part	Responsibility
Dashboard	The web interface (Next.js). The browser only talks to the dashboard's own server, which calls the gateway on the user's behalf. No token ever reaches client-side code.
Gateway	The single entry point. It receives GitHub webhooks, checks who is calling, serves the dashboard's API, and publishes results to GitHub (Check Run, comments, fix commits).
Orchestrator	Runs a review. A LangGraph graph plans the work, calls the agents in parallel with a timeout each, merges and ranks their findings, and looks up similar past issues.
Five agents	Security, Style, Performance, Logic and Documentation. Each is a separate service with the same API, so adding an agent needs no change elsewhere.
Database	Supabase PostgreSQL with the pgvector extension, used by the gateway and the orchestrator through Prisma. Supabase also handles sign-in.

Part	Responsibility
Shared packages	contracts (the API schemas every service imports), llm (LLM clients and quota budget), agent-kit and service-kit (shared service code) and db (schema, migrations and client).

2.2 How a review works

1. **GitHub sends a webhook** when a pull request is opened or updated. The gateway checks the signature, ignores repeats of the same delivery, and checks that the repository and its base branch are set to be reviewed.
2. **The gateway prepares the review.** It fetches the changed files, leaves out files that should not be reviewed (binary, generated, deleted, lockfiles), creates a Check Run that shows “in progress”, and hands the job to the orchestrator. GitHub gets its answer within seconds.
3. **The orchestrator plans and fans out.** It groups files that import one another, reserves LLM quota for each call, and calls the five agents in parallel. An agent that does not answer in time is recorded as timed out and the review continues without it.
4. **Each agent reviews.** Agents receive every changed file whole, with the pull request’s lines marked, and report only on added lines. The Style Agent runs real linters (ESLint, Prettier, Ruff, Black) and uses no AI; Security combines fixed rules with an AI pass; the other three use AI.
5. **The orchestrator combines the results.** Findings from different agents on the same lines and of the same kind become one finding that lists every agent. Findings are ranked by severity, and each is compared with earlier findings in the organization to add a “seen before” link.
6. **The gateway publishes.** It posts one inline comment per location with at most one committable fix, and completes the Check Run as pass, fail or neutral. A critical finding from an agent set to “Block merge” fails the check.
7. **The developer acts.** Fixes can be committed one at a time or all together in a single commit, from GitHub or from the dashboard. The new commit is reviewed again.

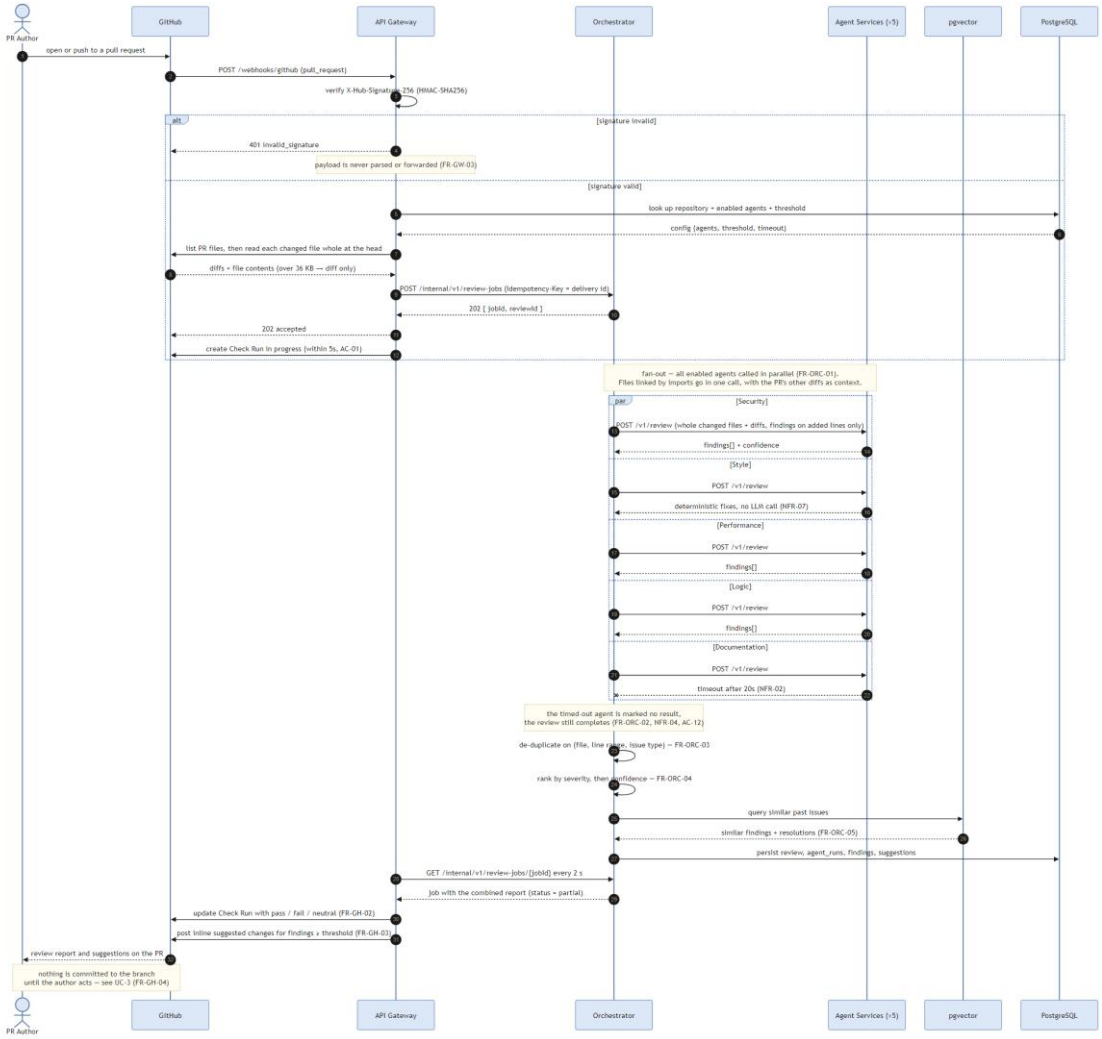


Figure 2. Reviewing a pull request, end to end

A review moves through a small set of states. A new push cancels the review of the older commit, and a review that loses an agent finishes as “partial” rather than failing.

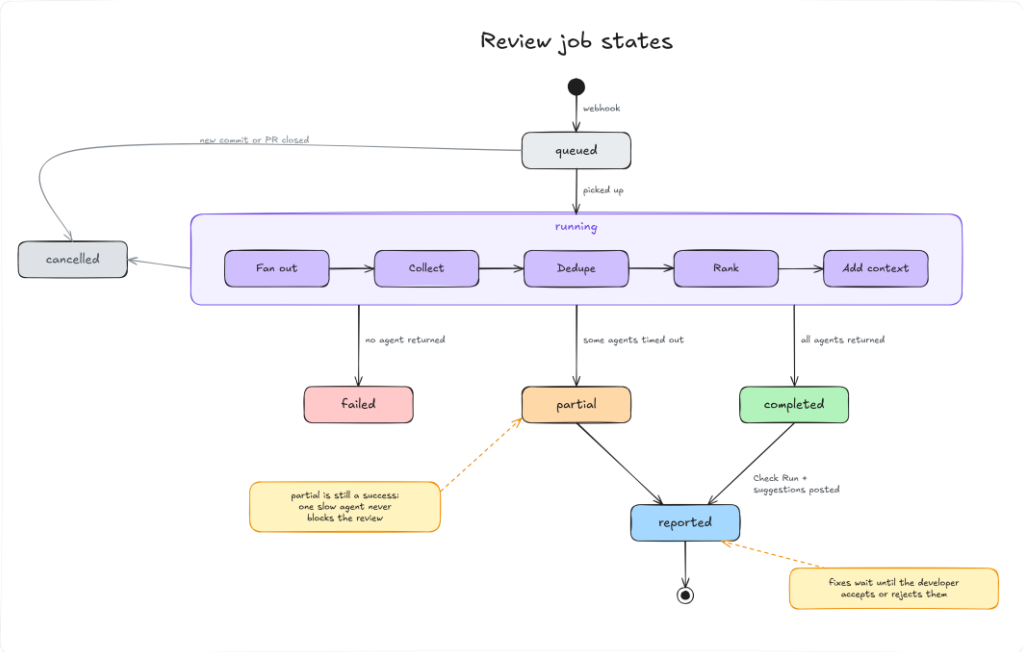


Figure 3. States of a review

2.3 Data model

The database has 18 tables in four groups: identity and organizations (users, members, GitHub installations and tokens); repositories and their settings (per-agent modes, thresholds, branches to review); reviews (reviews, agent runs, findings, suggestions, embeddings and similarities); and security records (device sessions and audit events).

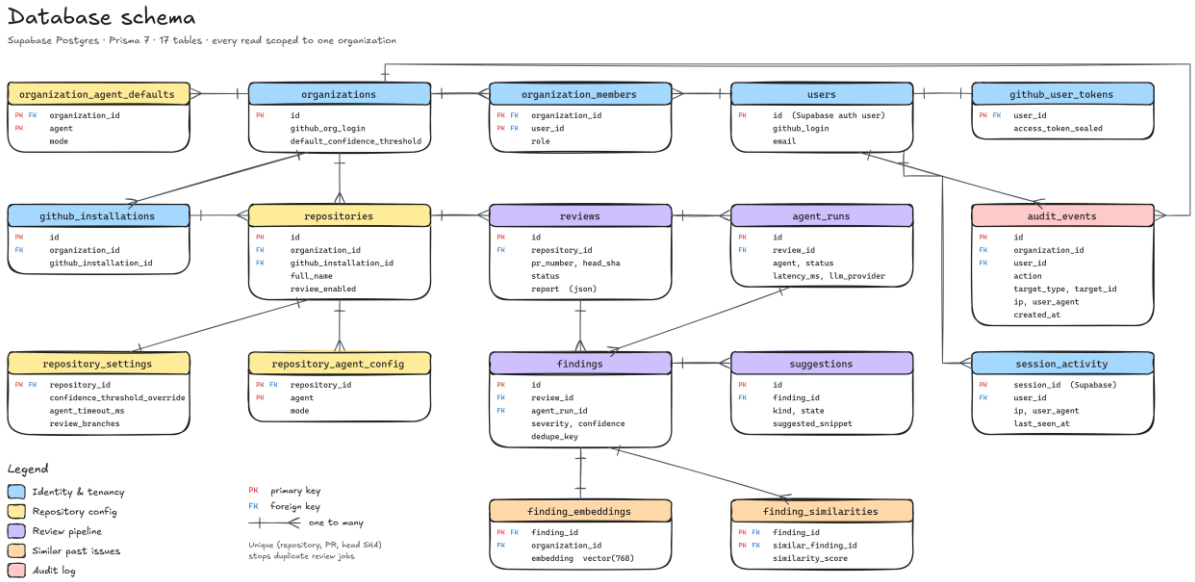


Figure 4. Database schema

Row-level security is switched on for every table with no policies, so nothing is readable with Supabase's public key; only the backend's own database role can reach the data. Every query is scoped to the caller's organization.

3. Technology Stack

Layer	Technology	Why
Language	TypeScript 5 on Node.js 22	One language across frontend, backend and shared contracts
Monorepo	npm workspaces, Turborepo	Shared packages, cached builds, one CI run
Frontend	Next.js 16, React 19, Tailwind CSS v4, shadcn/ui	Server-side calls keep tokens out of the browser
Gateway	Express 5, Octokit, jose, helmet, express-rate-limit	Small and well known; established libraries for auth and limits
Orchestrator	LangGraph	Parallel fan-out with timeouts and a clear graph of steps
Agents	Express on a shared agent kit; ESLint, Prettier, Ruff, Black	One contract for all agents; real linters for style
LLM	Groq (gpt-oss-120b), Google Gemini; Gemini embeddings	Free tiers; two providers so one can cover the other
Database	Supabase PostgreSQL, pgvector, Prisma 7	One database for data and vector search
Sign-in	Supabase Auth (email, GitHub, Google, TOTP)	Proven auth instead of a hand-written one
Contracts	OpenAPI 3.1, zod	Every request and response is checked against the written API
Testing	Vitest, Supertest	Fast unit and HTTP tests in every workspace
CI/CD	GitHub Actions, Render Blueprints, Vercel	Deploy only after checks pass
Hosting	Render (Docker), Vercel, Supabase	Free plans that fit a semester project
Monitoring	Prometheus metrics, Grafana Cloud	Dashboards and email alerts

Two choices changed during the project. The vector store moved from ChromaDB to pgvector, which removed a service that would have needed hosting. The LLM provider chain was reduced from four providers to two (Groq first, Gemini second), because the simpler chain made quota planning possible.

4. Features

4.1 Pull request review

- Automatic review on every pull request, with a Check Run that appears within seconds.
- Five agents in parallel, each with its own timeout; a slow or failed agent never stops the review.
- One comment per problem: findings from several agents on the same lines are merged and show who flagged them.
- One-click fixes on each comment, and **Apply all fixes** to commit every fix in one commit.
- “Seen before” links to similar earlier findings in the same organization.
- Large pull requests are reviewed as far as possible; skipped files are listed with the reason.
- Repeated webhook deliveries are ignored, and a new push cancels the older review.

4.2 Dashboard

- Sign in with email and password, GitHub or Google; password reset by email.
- Optional authenticator app. Once added, changing settings, connecting GitHub, committing fixes and signing out a device ask for a code.
- Connect repositories by installing the GitHub App; repositories sync automatically.
- Per repository: reviews on or off, each agent Off / Comment / Block merge, a confidence threshold with a live preview, and the branches to review.
- Organization-wide defaults that repositories can follow.
- Overview of what is blocked or running, with agent health notices.
- Review page with live progress while agents run, findings by file, and Commit fix, Dismiss, Apply all fixes and Re-run review.
- Signed-in devices with sign-out for each, and an audit log for administrators.
- Members see only the repositories they can open on GitHub.

4.3 Operations

- Rate limits per GitHub installation, per user and per IP address.
- An audit record for every sensitive action, with user, IP address and browser.
- Metrics from the gateway and orchestrator, a Grafana dashboard and four alerts.
- LLM quota reserved before every call, so free-tier limits are never exceeded.

5. Challenges and Solutions

5.1 Staying inside free LLM quotas

Problem. The agents depend on free LLM tiers. Groq allows only 8,000 tokens a minute on the chosen model, and one review with four AI agents can need far more. Early runs received “429 Too Many Requests” errors, which left reviews incomplete.

Solution. The orchestrator now plans before it calls. It estimates the tokens of each batch and reserves them against a per-provider budget that tracks requests and tokens per minute and per day, using 80% of each limit. A batch goes to Groq if Groq can take it now, otherwise to Gemini, otherwise it waits. After the call, the reservation is corrected with the provider’s real usage. If a Groq call still fails, Gemini retries it once. An agent that cannot get quota before the deadline is reported as “deferred”, and the review is marked partial instead of failing.

5.2 Too little context from diffs

Problem. The first design sent agents only the changed lines. Linters reported false syntax errors on fragments cut in the middle of a function, and the AI agents missed problems that depend on the surrounding code.

Solution. Agents now receive each changed file whole, with the pull request’s lines marked, and report only on those lines. Files that import one another are reviewed in the same call, and every call also sees the rest of the pull request’s changes. Files that are too large are sent as a diff, and the ones that still do not fit are listed as skipped, so the developer always knows what was and was not reviewed.

5.3 Duplicate findings from different agents

Problem. With five agents, the same problem was often reported two or three times, for example a database query in a loop flagged by both Performance and Logic. Pull requests filled with repeated comments.

Solution. The orchestrator merges findings that overlap in lines and belong to the same family of issue. The merged finding keeps the highest severity and the best explanation and names every agent. The gateway posts one comment per location with one fix, chosen by priority, and mentions the other suggestions as text.

5.4 Running linters where Docker is not available

Problem. The Style Agent was designed to run linters inside a locked-down Docker container for each request. Render, the hosting platform, does not allow a service to start containers.

Solution. The agent gained a second mode. On Render the linters are installed in the agent’s own image and run as a child process: a non-root user, a temporary folder per request, a time limit and a memory cap, one run at a time. This is acceptable because the agent holds no secrets and never executes the code it checks. Local development keeps the Docker sandbox.

5.5 Free hosting that sleeps

Problem. Free Render services sleep after 15 idle minutes and take about a minute to wake, but GitHub waits only about ten seconds for a webhook answer. Free hours are also shared within one workspace, and seven services would exhaust them.

Solution. The services are split over three workspaces. The gateway and the orchestrator each have their own and are kept awake by a scheduled request every ten minutes. The five agents share the third and are allowed to sleep; the orchestrator wakes each one before calling it, and the dashboard explains the wait while a review runs. Monitoring does not poll the agents, since that would keep them awake and use up their hours.

5.6 Finding the real user behind proxies

Problem. Rate limits and the audit log need each user's real IP address. In production every request passes through several proxies, and the first attempts recorded a proxy's address for every user, so all users shared one rate limit.

Solution. The number of proxy hops was measured on the live system (four, including the dashboard's own server) and configured in the gateway. The dashboard forwards the browser's address, and the gateway records the device behind each session itself, because Supabase only ever sees the dashboard's server.

5.7 Sign-out that takes effect immediately

Problem. Access tokens stay valid until they expire, so a user who signed out, or whose device was signed out, could keep calling the API for several minutes.

Solution. After verifying a token's signature, the gateway also checks that its session still exists. Supabase's own tables cannot be opened to the backend's database role, so the check goes through a few narrow database functions that only that role may run.

5.8 Keeping eight people's work consistent

Problem. Seven services written by different people must agree exactly on the data they exchange.

Solution. The API is written first in OpenAPI and mirrored as schemas in a shared package that every service imports. Requests and responses are validated against those schemas at run time, and the in-memory and database versions of each store are tested with the same shared test suite, so they cannot drift apart.

6. Screenshots

6.1 On GitHub

The review appears on the pull request itself. The Check Run summarises the verdict, what each agent found and how much of the change was reviewed; each problem gets one inline comment with a fix that can be committed directly.

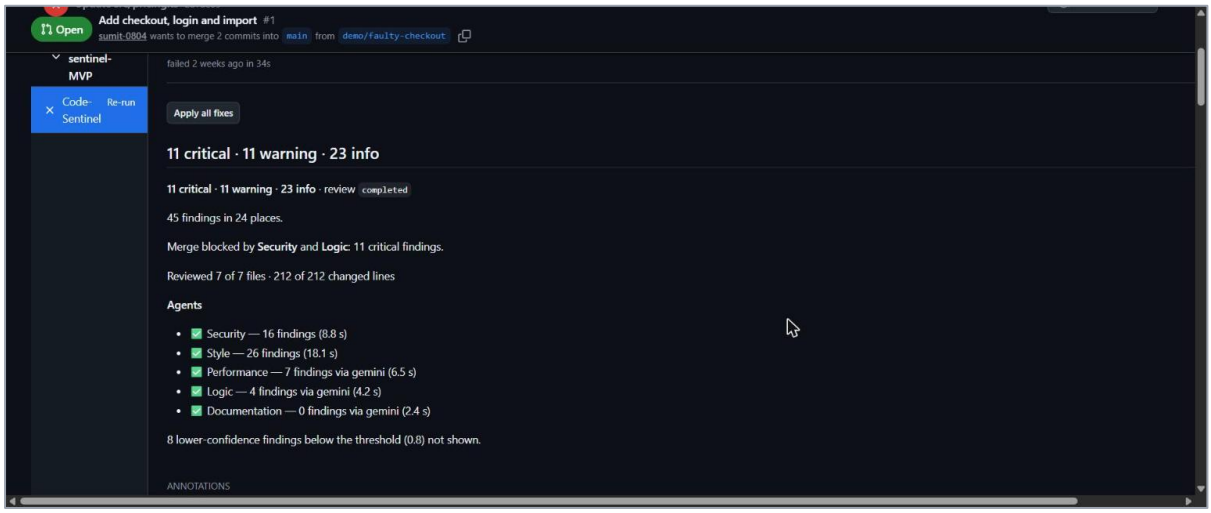


Figure 5. The Code-Sentinel Check Run: verdict, agents, coverage and Apply all fixes

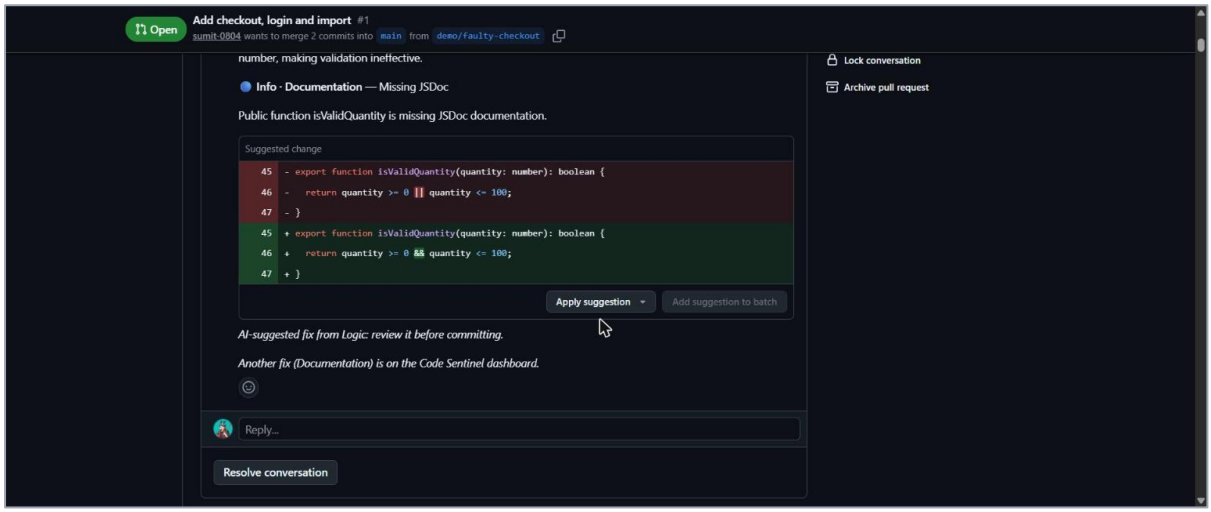


Figure 6. An inline comment with a suggested fix that can be committed in one click

6.2 Dashboard

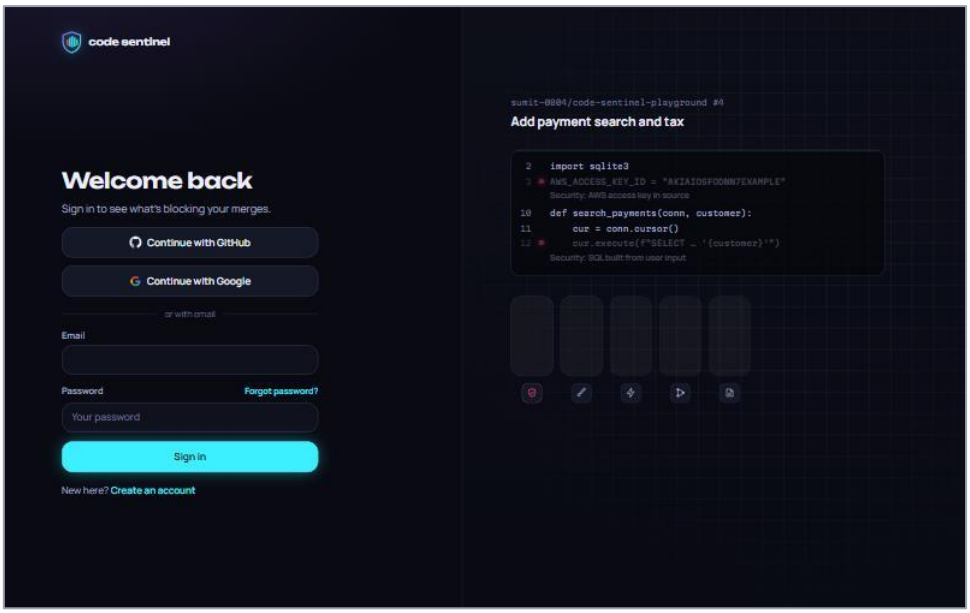


Figure 7. Sign-in with GitHub, Google or email and password

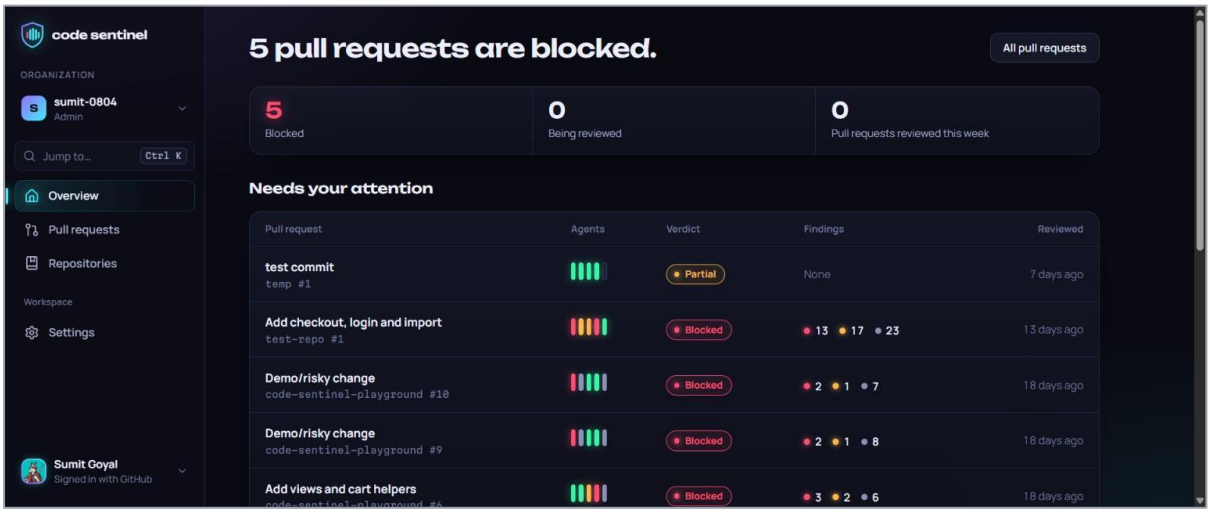


Figure 8. Overview: blocked pull requests and what needs attention

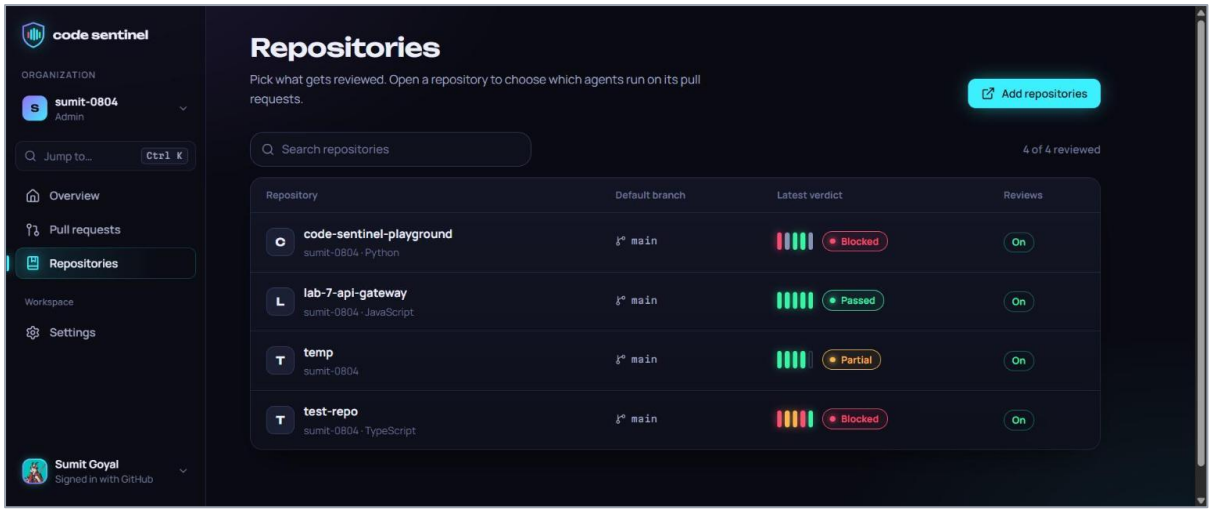


Figure 9. Connected repositories with their latest verdict

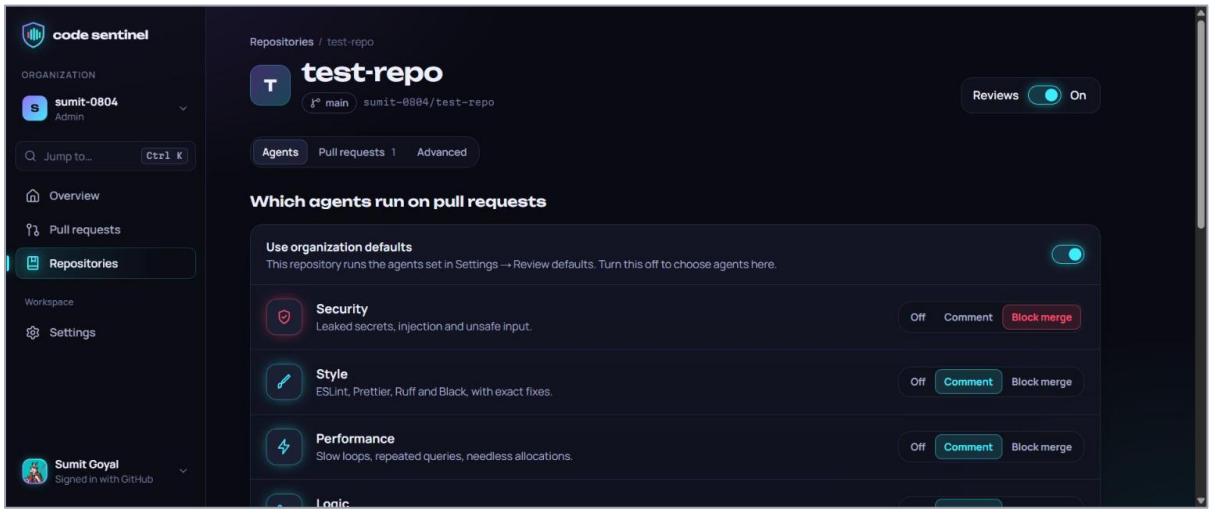


Figure 10. Per-repository agent modes: Off, Comment or Block merge

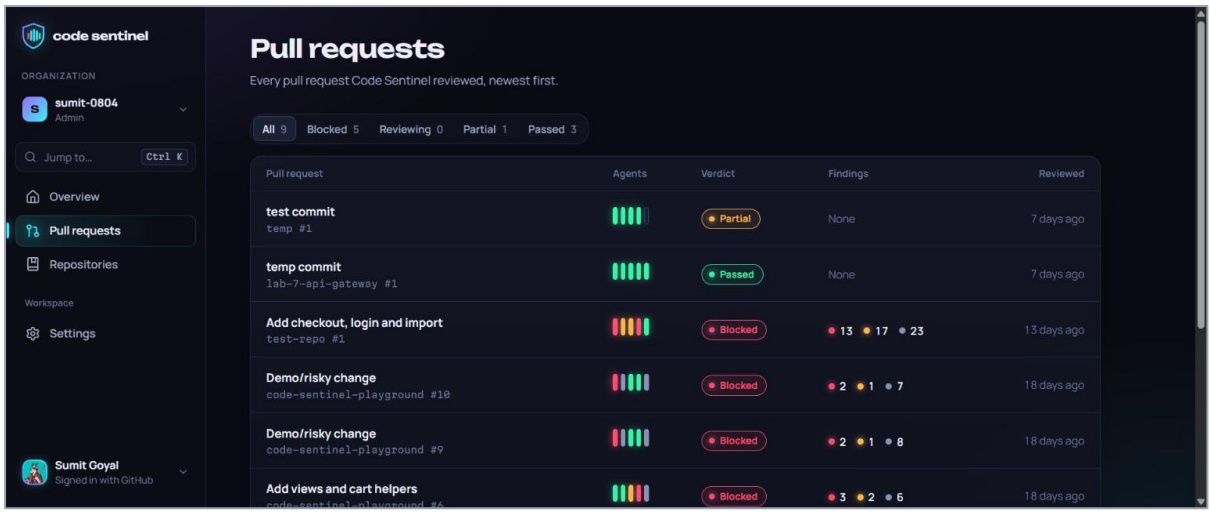


Figure 11. Pull requests with each agent's result, verdict and findings by severity

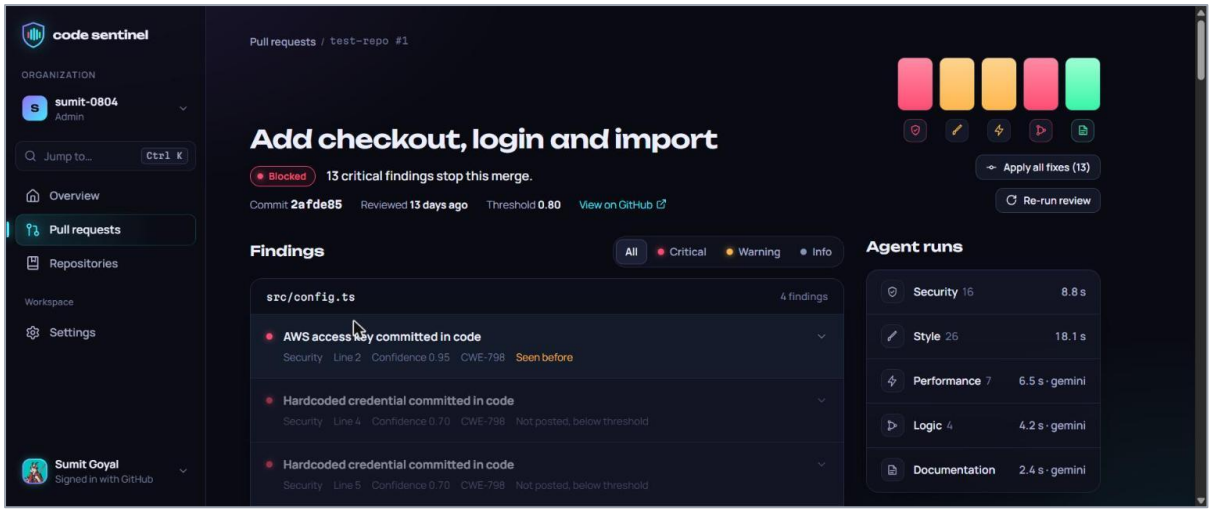


Figure 12. A review: verdict, agent runs, Apply all fixes and Re-run review

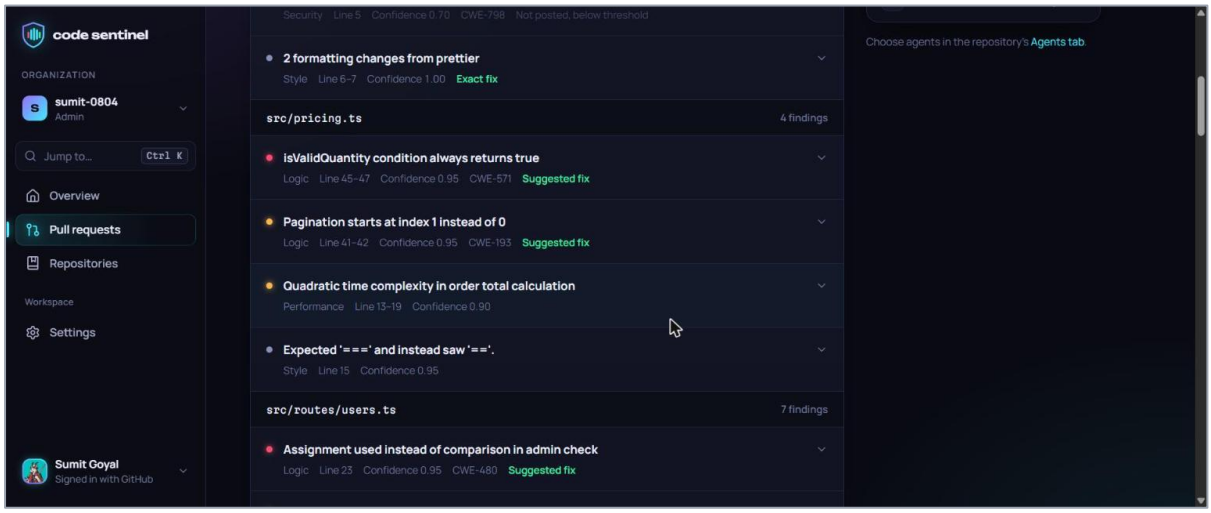


Figure 13. Findings grouped by file, with exact and AI-suggested fixes

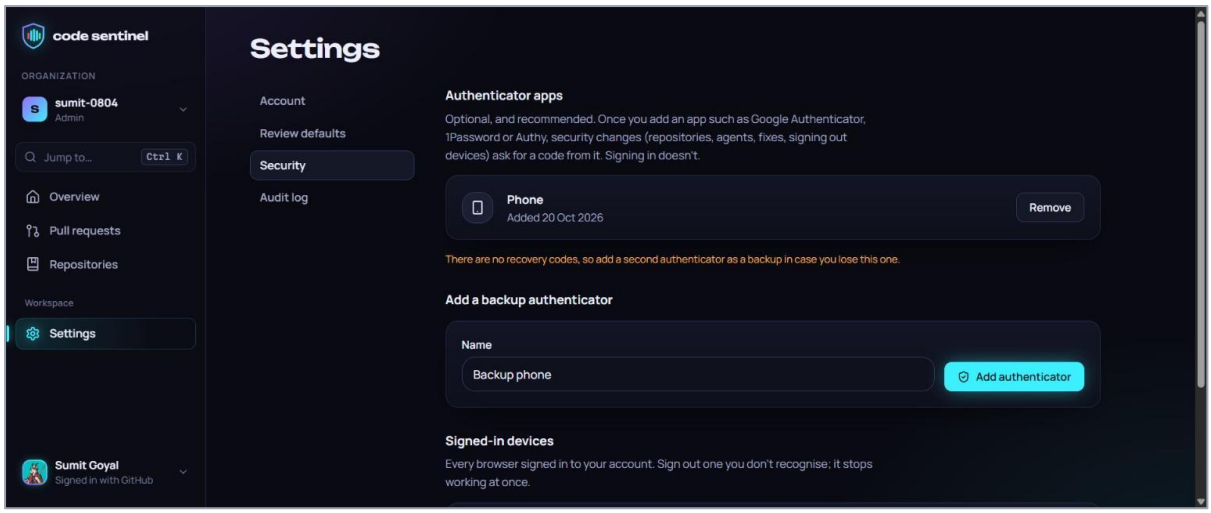


Figure 14. Security settings: authenticator apps and signed-in devices

7. Testing

Testing is described in full in the Test Plan and Test Cases document in the repository (docs/CodeSentinel_Test_Plan_and_Test_Cases.pdf). This section summarises it.

7.1 Approach

Level	What is tested
Unit	Pure logic in every workspace: diff parsing, security rules, quota budget, token planning, merging and ranking of findings, encryption, configuration loading.
Service and API	Each service through HTTP: authentication, roles, two-factor step-up, rate limits, webhook handling, and every API route against its contract.
Integration	Gateway to orchestrator to agents with stubbed GitHub and LLM calls: full review, cancelled reviews, timeouts, fixes and tenant isolation.
Database	Migrations applied to a fresh PostgreSQL with pgvector; constraints, unique indexes, cascades, vector search and the seed data in all 18 tables.
Live	Real pull requests on test repositories with seeded bugs for every agent, on the deployed system.
Non-functional	Interface, performance, reliability, security and monitoring checks.

7.2 Results

- **868 automated tests** in 103 test files across 13 workspaces pass on Node.js 22 and 24. LLM and GitHub calls are stubbed in tests, so no test uses a key or the network.
- **Continuous integration** runs lint, build, type checking, database migrations and all tests on every push. A failed run blocks deployment.
- **22 defects** were logged during development and testing and all are closed. Examples: an LLM fallback call that ignored the review's deadline, duplicate findings a line apart, and a proxy address recorded instead of the user's.
- **Live runs.** The complete flow was run on real pull requests: Check Run within seconds, inline comments on the correct lines, one-click and apply-all fixes, re-review of the new commit, and a repeated webhook ignored. The seeded-bug pull request in Section 6 was reviewed in 34 seconds: 45 findings in 24 places, all 7 files covered.

7.3 Demo data

A seed script fills all 18 tables with realistic, connected records (two organizations, reviews in every state, merged findings, accepted and dismissed fixes, similar-issue links, devices and audit events). It can be run repeatedly without creating duplicates and is refused in production.

8. Deployment

The deployment notes are in the repository at docs/DEPLOYMENT.md. The figure below shows where each part runs.

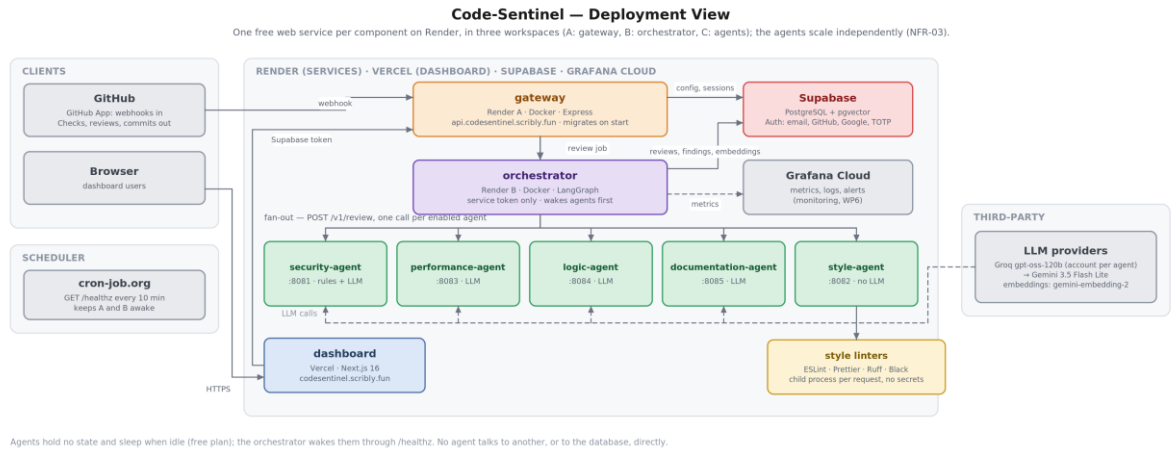


Figure 15. Deployment view

8.1 Hosting

Part	Where	Address
Dashboard	Vercel	https://codesentinel.scribly.fun
Gateway	Render (Docker)	https://api.codesentinel.scribly.fun
Orchestrator	Render (Docker)	called only by the gateway
Five agents	Render (Docker)	called only by the orchestrator
Database and sign-in	Supabase	–
Email	Resend, through Supabase	–
Monitoring	Grafana Cloud	–
DNS	Hostinger	–

8.2 CI/CD

- **CI** (GitHub Actions): clean install, lint, build, type check, database migrations on a fresh PostgreSQL, and all tests, on Node.js 22 and 24.
- **Deployment:** Render redeploys a service only after CI passes on the main branch and only when that service's files changed. Each service has a health check. Vercel builds the dashboard from the same branch.
- **Database:** the gateway applies pending migrations before it starts.

8.3 Security configuration

- HTTPS only, with HSTS, on both domains.

- Every API call carries a short-lived token that the gateway verifies against Supabase's public keys, and whose session must still be signed in.
- Optional two-factor authentication, required again before sensitive changes.
- Administrator role for configuration and the audit log; members limited to their own repositories.
- GitHub webhooks accepted only with a valid signature; calls between services need a shared service token.
- Rate limits on webhooks, users, writes and failed sign-ins.
- Row-level security on every table; GitHub user tokens encrypted at rest.
- Secrets kept only in the hosting platforms' settings, never in the repository.
- Security headers (Content Security Policy, frame and content-type protection) on every page; services run as a non-root user.

8.4 Monitoring

The gateway and orchestrator expose Prometheus metrics behind a token. Grafana Cloud reads them every minute and shows requests, errors, latency, webhooks, rate limits, agent runs and health, and remaining LLM quota. Four alerts send email: a service down, an agent failing, a high error rate, and LLM quota running low.

9. Team and Contributions

Member	Student ID	Main contributions
Sumit Goyal	202512092	Orchestrator graph, parallel fan-out and timeouts, LLM layer and quota routing, project coordination and reports
Durgesh	202512052	Merging, de-duplication and ranking of findings, pgvector context and embeddings
Tej Prakash Tak	202512113	Security Agent and Logic Agent, gateway rate limits
Nevil Nandasana	202512009	Style Agent and its sandbox, service metrics
Parin Makwana	202512098	Performance Agent and Documentation Agent, sessions API
Ayush Soni	202512030	Gateway, database schema and migrations, authentication, audit log
Vatsal Kamani	202512068	Monorepo and CI, GitHub App integration, deployment and monitoring
Moksh Mehta	202512120	Dashboard: all screens, sign-in flows, live review progress

All eight members committed code throughout the project. Progress was reported in bi-weekly status reports, which are in the repository.

Limitations and future work

- **Cold starts.** On free hosting the first review after a quiet period takes about a minute longer while agents wake. Paid instances would remove this.
- **One gateway instance.** Rate-limit counters live in memory. Running several instances would need a shared store such as Redis.
- **Languages.** Style checks cover JavaScript, TypeScript and Python. More linters can be added behind the same agent.
- **Learning from decisions.** Dismissed fixes are recorded with a reason but are not yet used to tune what agents report.
- **More clients.** Only GitHub is supported. The gateway's design leaves room for other code hosts.

Conclusion

Code Sentinel met its goal: a deployed, working platform in which five independent agents review real pull requests and return one clear, de-duplicated review with fixes that can be applied in a click. The service-oriented design held up in practice. Agents were added without changing the rest of the system, the vector store and the LLM providers were replaced mid-project behind stable contracts, and the whole system runs within free hosting and LLM limits.